SYSTEM REQUIREMENTS SPECIFICATION (SRS)

CLICK COMPUTER AND ACCESSORIES

ITC327W – Systems Analysis and Design

Version 4.0 – Unit 1, 2, 3 and 4

1 November 2026

Prepared by:

Name	Student Number	Role
Lesako M	221007662	Project Manager
Hlalele D	221003431	SRS and Requirement Analyst
Tladi K	221010874	Stakeholder Analyst
Taaibosch K	222001440	Problem Analyst
Molefe B	223019042	Feasibility Analyst
Ndlovu N	223038645	Risk Manager
Mafunisa T	222033930	Technical Architecture
Lesenyeho LJ	223002326	Project Planner
Winkel K	221011353	Documentation and Quality Controller
Nosenga B	223066258	GitHub and Presentation Lead

PHASE 1

Engagement, Requirements, Feasibility and Risk

Units 1 and 2

1. INTRODUCTION

1.1 Stakeholder and Organisation

Click Computer and Accessories provides computer and electronic support services to clients. Clients may require assistance with cellphones, laptops, printers, televisions and other electronic equipment.

The proposed system is intended to improve how support requests are submitted, monitored, assigned and tracked hence a need for a centralised support platform.

The main stakeholders are clients, support operators, technicians and administrators/managers. The existing process may depend on direct communication with staff, making it difficult to track requests, assign work consistently and maintain visibility of progress.

1.2 Refined Problem Statement

The main problem is the lack of a simple, centralised platform through which clients can select their device, identify the problem or service required, submit a support request and track its progress. The current approach may result in delays, missed requests, unclear technical information, difficulty assigning technicians and limited visibility of request progress.

1.3 Project Aim and Objectives

Aim: To develop a simple client-support platform for Click Computer and Accessories that enables clients to submit, communicate about and track computer and electronic support requests.

Objectives:

1. Provide a simple and user-friendly client application.
2. Allow clients to register and log in, select a supported device, select a relevant problem/service, describe the problem, submit a request and track its status.
3. Allow authorised support operators to monitor incoming requests.
4. Allow support operators to communicate with clients.
5. Allow support requests to be assigned to technicians.

6. Allow administrators to manage users, requests and assignments.
7. Provide notifications for important request events where retained in the final scope.
8. Allow relevant device or asset information to be recorded where required.

1.4 Project Scope

Included Scope

- Client registration and login.
- Device selection.
- Problem/service selection.
- Support-request submission.
- Request status tracking.
- Client/support communication.
- Technician assignment.
- Administrator controls.
- Device/asset information where required.
- Supporting evidence where retained.

Exclusions

- Unapproved device categories.
- Complex financial/accounting functions.
- Advanced artificial-intelligence diagnosis.
- Automated physical repair of devices.
- Unauthorised access to client devices.
- Features outside agreed stakeholder requirements unless approved through change control.

1.5 Constraints and Assumptions

Technical: Flutter/Dart frontend, C# and ASP.NET Core 6.0 backend, and Supabase persistent storage.

Time: four-week development cycle.

Budget: R0/free-tool approach as stated in the project information. Users are assumed to have suitable internet access, provide accurate information, and use the system according to their roles.

1.6 Deliverables

Project Deliverables

1. Flutter client application.
2. ASP.NET Core backend/API.
3. Supabase database.
4. Client support-request functionality.
5. Request tracking and communication functionality.
6. Technician assignment functionality.
7. Administrator management functionality.

2. STAKEHOLDERS, USERS AND CURRENT PROCESS

2.1 Stakeholders and Users

Stakeholder/User	Role
Clients	Submit and track support requests.
Technicians	Handle technical problems assigned to them.
Administrators/Managers	Manage users, requests, assignments and service operations.

2.2 Current Process

The existing process may require a client to communicate directly with staff, explain the device and problem manually, wait for the request to be handled and receive feedback through staff communication.

2.3 Current Process Problems

The identified problems include delays in receiving support, difficulty communicating technical problems clearly, requests being missed or forgotten, difficulty assigning requests to appropriate technicians, limited visibility of request progress, increased workload for support staff and poor communication.

2.4 Proposed Process

Client opens application → selects device → selects problem/service → describes and submits request → support operator monitors request → request is assigned to technician where necessary → technician resolves the problem → status is updated → client tracks progress.

3. SYSTEM REQUIREMENTS

3.1 Functional Requirements

ID	Requirement	Priority
FR1	The system shall allow clients to register and log in.	Must Have
FR2	The system shall provide an approved device menu for clients.	Must Have
FR3	The system shall display relevant problem/service options after a device is selected.	Must Have
FR4	The system shall allow clients to select a problem, enter a description and submit a support request.	Must Have
FR5	The system shall allow clients to view the current status of a submitted request.	Must Have
FR6	The system shall provide communication between clients and authorised support staff regarding support requests.	Must Have
FR7	The system shall allow authorised users to assign support requests to appropriate technicians.	Must Have
FR8	The system shall allow authorised administrators to manage users, requests and assignments.	Must Have
FR9	The system should send notifications for important request events.	Should Have
FR10	The system should allow relevant device or asset information to be recorded.	Should Have
FR11	The system should allow relevant screenshots, photographs or documents to be attached.	Should Have

3.2 Non-functional Requirements

ID	Requirement	Priority
NFR1	The system shall protect client, technician and administrator information from unauthorised access.	Must Have
NFR2	The system shall provide clear confirmation, status and error messages.	Must Have
NFR3	The system shall maintain accurate and consistent support-request information.	Must Have
NFR4	The client application shall use clear navigation and require minimal steps for submitting a request.	Must Have
NFR5	API responses shall return a result within 2 seconds under normal load.	Must Have

3.3 System/Technical Requirements

ID	Requirement	Priority
SR1	The frontend shall be developed using Flutter and Dart.	Must Have

ID	Requirement	Priority
SR2	The backend shall be developed using ASP.NET Core 6.0 and C#.	Must Have
SR3	The system shall use Supabase for persistent data storage.	Must Have

3.4 Integration Requirements

ID	Requirement	Priority
IR1	Flutter shall communicate with the ASP.NET Core backend through APIs.	Must Have
IR2	ASP.NET Core shall communicate with Supabase to store and retrieve relevant data.	Must Have
IR3	Client support-request information shall pass through the backend and be stored in the database.	Must Have
IR4	Authorised request-status changes shall be reflected in the client application.	Must Have
IR5	Communication and notification events shall be connected to relevant requests and users where implemented.	Should Have

3.5 Requirements Traceability Table

Stakeholder Need	Requirements
Clients need to submit support request	FR1, FR2, FR3, FR4
Clients need to know request status	FR5
Clients need to communicate with support staff	FR6
Support staff need to assign technician	FR7
Admins need to manage requests	FR8
Users need secure access	NFR1
Users need simple navigation	NFR4

4. FEASIBILITY ANALYSIS

Feasibility Area	Decision	Main Reason	Change Required
1. Technical	Feasible	Separate Flutter/Dart frontend, ASP.NET Core 8.0 backend, and Supabase.	Mitigate integration risks strictly within the restricted development timeline.

Feasibility Area	Decision	Main Reason	Change Required
2. Economical	Feasible	The project will primarily use free/open-source development tools and available services. This supports the project's R0 budget constraint. A limitation is that some hosted services may have usage restrictions or paid tiers.	The team will prioritise free service tiers and replace any paid dependency if it becomes necessary.
3. Operational	Feasible	The proposed system is suitable for Click Computer and Accessories because it centralises support requests, allows support operators to monitor requests and enables clients to track progress.	The main limitation is that staff and clients will need to become familiar with the new workflow. Short user training and clear instructions will therefore be required.
4. Schedule	Feasible	Significant features can be delivered in a realistic timeline using Microsoft Project Schedule.	We consider using the MS Project schedule to track progress, milestones and avoid feature creep.
5. Legal	Feasible	System will handle personal data and support conversations through notifications or emails.	Implementing proper access controls and data-retention policy. Legal review of privacy notice will be recommended.
6. Ethical	Feasible	System is used for legitimate operations. Main risks are misuse of data and lack of transparency.	Defining clear usage policy, role-based access, and monitoring that the system is used fairly.

4.2 Overall Feasibility Decision

The Click Computer and Accessories support-request system is considered feasible with changes. The technical, operational, economic, schedule, legal and ethical feasibility assessments indicate that the project can be developed using the proposed Flutter, ASP.NET Core and Supabase technologies. However, changes were required to keep the project realistic within the available time, resources and project scope. These changes include prioritising core support-request functionality, moving simplicity to the non-functional requirements, treating 24/7 active management as an operational requirement, and retaining notifications, asset/device information and supporting evidence as lower-priority Should Have requirements. The refined scope will therefore focus on the core functions required for clients, support staff, technicians and administrators.

5. PROJECT RISKS

5.1 Risk Register

Risk ID	Risk Statement	Likelihood	Impact	Score	Level	Mitigation	Contingency	Owner	Status	Category
R01	Clients may struggle to use the interface and submit support requests.	3	3	9	High	Keep the main client journey simple and easy to navigate.	Provide guidance to users and simplify the affected screens if required.	Taaibosch K	Active	Operational

Risk ID	Risk Statement	Likelihood	Impact	Score	Level	Mitigation	Contingency	Owner	Status	Category
R02	Support requests may be missed or not handled on time.	2	3	6	Medium	Provide active monitoring and notifications where implemented.	Review missed requests and assign them to an available support operator.	Ndlovu N	Active	Operational
R03	A support request may be assigned to the wrong technician.	2	3	6	Medium	Use authorized assignment according to the device and problem.	Reassign the request to the appropriate technician.	Mafunisa T	Active	Operational
R04	Poor communication may result in clients not receiving progress information.	2	3	6	Medium	Provide communication linked directly to support requests.	Contact the client through an alternative approved communication method.	Molefe B	Active	Communication
R05	Clients may provide incomplete technical information about their problem.	3	2	6	Medium	Allow supporting evidence such as screenshots, photographs or documents where implemented.	Contact the client and request the missing information or evidence.	Taaibosch K	Active	Operational
R06	Incorrect request status information may mislead clients.	2	3	6	Medium	Restrict status changes to authorized users.	Correct the status and inform the affected client of the correct progress.	Hlalele D	Active	System
R07	Flutter, ASP.NET Core and Supabase components may fail to communicate correctly.	2	3	6	Medium	Test Flutter, ASP.NET Core and Supabase throughout development.	Identify the failed component and restore communication before continuing development.	Mafunisa T	Active	Technical
R08	Limited development time may result in some requirements not being completed.	3	3	9	High	Prioritize Must Have requirements before lower-priority features.	If a Must-Have feature is not ready by the end of Week 3, the team will inform the Project Manager immediately to re-prioritize the remaining features.	Lesako M	Active	Schedule
R09	The project budget may not support commercial services.	2	2	4	Low	Use free tools and services where possible.	Remove or replace features that require paid services.	Lesako M	Monitoring	Economical
R10	Continuous 24/7 support monitoring may not be maintained.	2	2	4	Low	Designate an active support operator if 24/7 monitoring remains required.	Adjust monitoring responsibilities or operating hours if continuous coverage cannot be maintained.	Nosenga B	Monitoring	Operational

6. REQUIREMENTS CHANGE LOG AND REFINED SCOPE

6.1 Requirements Change Log

Area	Decision	Reason
Simple client application	Moved to non-functional usability requirements.	Simplicity describes quality rather than a business function.
Laptop repair examples	Removed as a standalone requirement.	Too specific; covered by problem/service selection.
24/7 active management	Moved to operational requirements.	Continuous staffing is an organisational responsibility.
Notifications	Retained as Should Have.	Useful but lower priority than core request processing.
Asset/device information	Retained as Should Have.	Relevant but lower priority.
Supporting evidence	Retained as Should Have.	Useful for technical diagnosis but can be deferred.

6.2 Refined Project Scope

The final scope prioritises client registration/login, device selection, problem/service selection, request submission, request tracking, communication, technician assignment and administrator management. The system will use Flutter/Dart, ASP.NET Core 6.0/C# and Supabase.

Notifications, asset/device information and supporting evidence will be implemented only if time and resources allow.

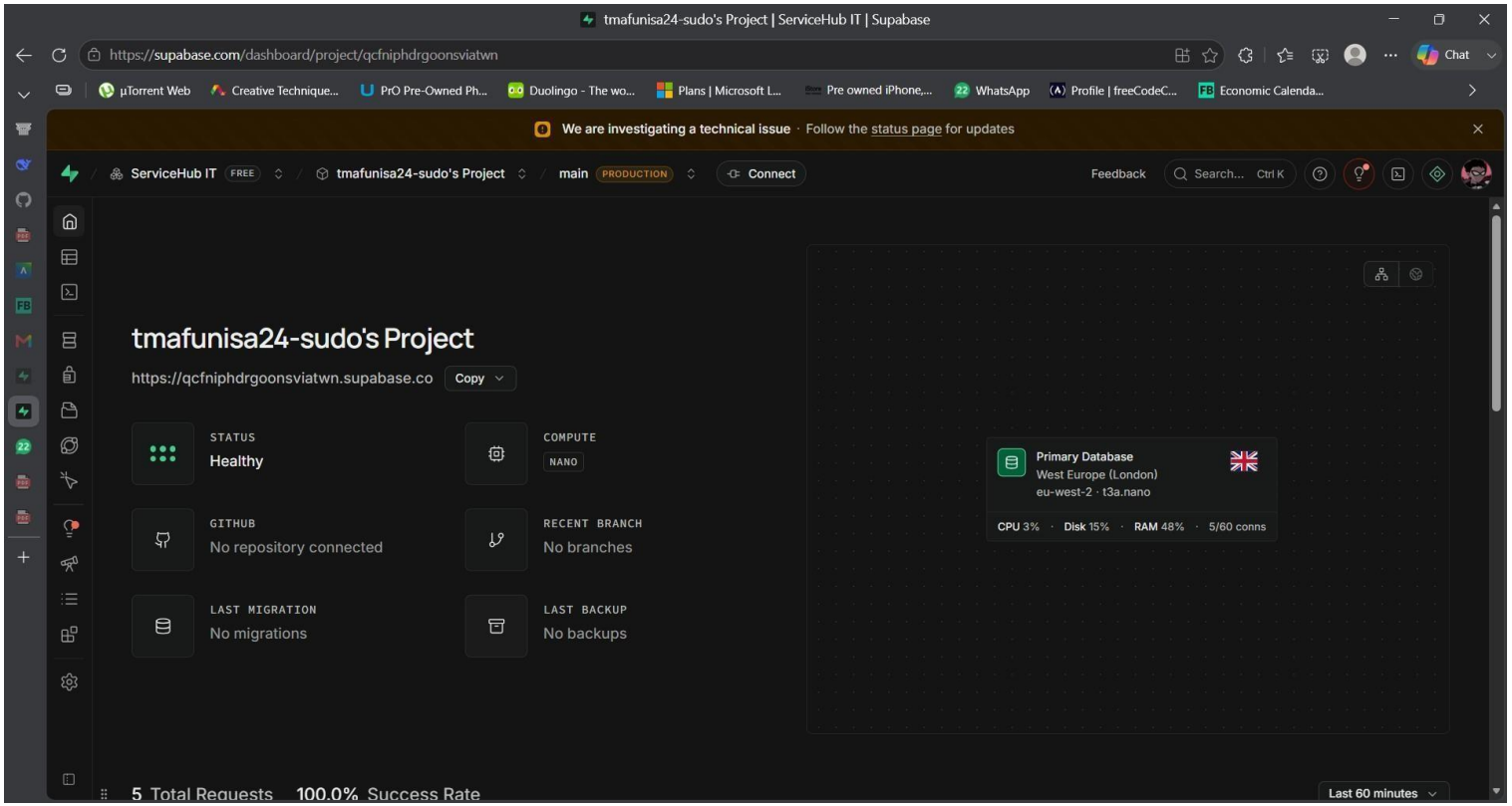
7. STAKEHOLDER REGISTER

Names	Type	Roles
1. Prince Morafo	Main Stakeholder – External	Project Stakeholder
2. Lesako M	Internal	Project Manager
3. Hlalele D	Internal	Requirements Analyst
4. Tladi K	Internal	Stakeholder Analyst
5. Taaibosch K	Internal	Business/Problem Analyst
6. Molefe B	Internal	Feasibility Analyst
7. Ndlovu N	Internal	Risk Manager
8. Mafunisa T	Internal	Solution/Technical Architect

Names	Type	Roles
9. Lesenyeho LJ	Internal	Project Planner
10. Winkel K	Internal	Documentation and Quality Controller
11. Nosenga B	Internal	GitHub and Presentation Lead

8. SUPABASE PROJECT SETUP

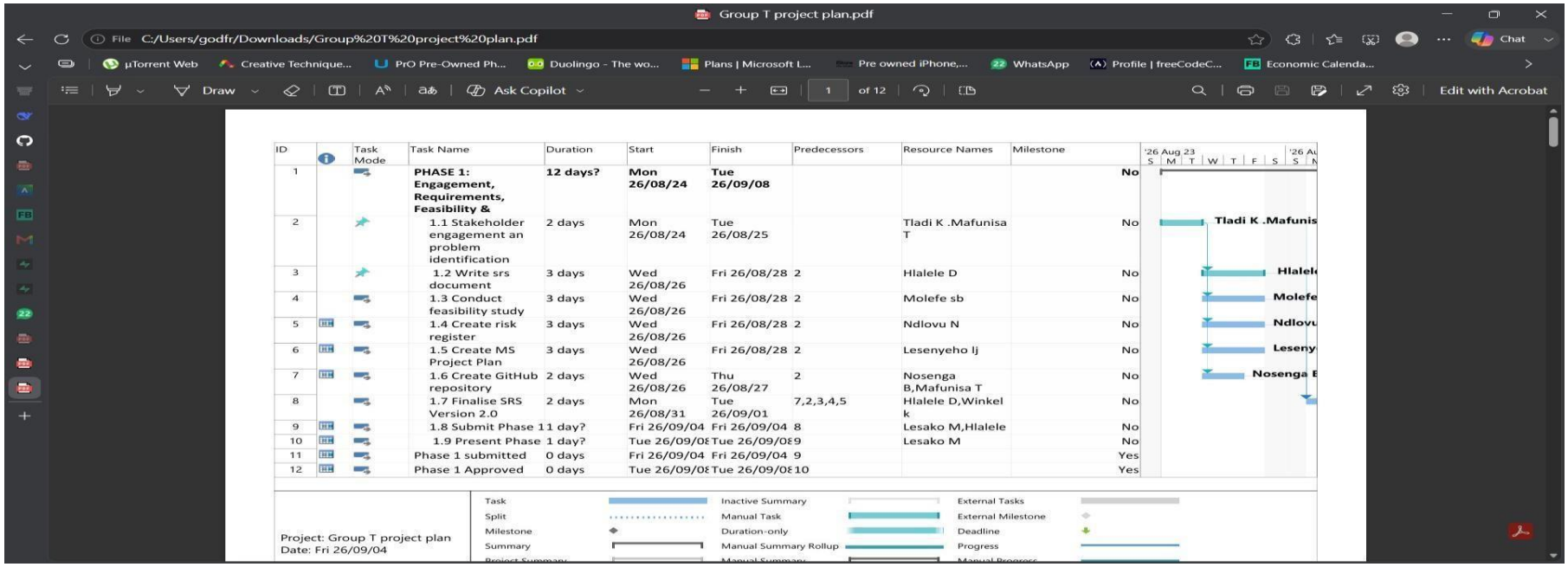
Supabase Project Dashboard



Supabase Project Details:

Detail	Information
Project Name	Click Computer and Accessories
Database Type	PostgreSQL
Status	Healthy
Authentication	Supabase Authentication
Storage	Available for future use

9. MICROSOFT PROJECT



ID	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
1		**PHASE 1: Engagement, Requirements, Feasibility &**	**12 days?**	**Mon 26/08/24**	**Tue 26/09/08**			**No**
2		1.1 Stakeholder engagement an problem identification	2 days	Mon 26/08/24	Tue 26/08/25		Tladi K .Mafunisa T	No
3		1.2 Write srs document	3 days	Wed 26/08/26	Fri 26/08/28	2	Hlalele D	No
4		1.3 Conduct feasibility study	3 days	Wed 26/08/26	Fri 26/08/28	2	Molefe sb	No
5		1.4 Create risk register	3 days	Wed 26/08/26	Fri 26/08/28	2	Ndlovu N	No
6		1.5 Create MS Project Plan	3 days	Wed 26/08/26	Fri 26/08/28	2	Lesenyeho lj	No
7		1.6 Create GitHub repository	2 days	Wed 26/08/26	Thu 26/08/27	2	Nosenga B,Mafunisa T	No
8		1.7 Finalise SRS Version 2.0	2 days	Mon 26/08/31	Tue 26/09/01	7,2,3,4,5	Hlalele D,Winkelk	No
9		1.8 Submit Phase 1	1 day?	Fri 26/09/04	Fri 26/09/04	8	Lesako M,Hlalele	No
10		1.9 Present Phase	1 day?	Tue 26/09/0	Tue 26/09/0	9	Lesako M	No
11		Phase 1 submitted	0 days	Fri 26/09/04	Fri 26/09/04	9		Yes
12		Phase 1 Approved	0 days	Tue 26/09/0	Tue 26/09/0	10		Yes

Milestone Summary:

Milestone	Target Date	Status
Stakeholder Confirmed	25 August 2026	Completed

Milestone	Target Date	Status
Requirements Draft	25 August 2026	Completed
Feasibility and Risk Done	30 August 2026	Completed
Full Draft Review	02 September 2026	Completed
Final Submission	04 September 2026	Completed

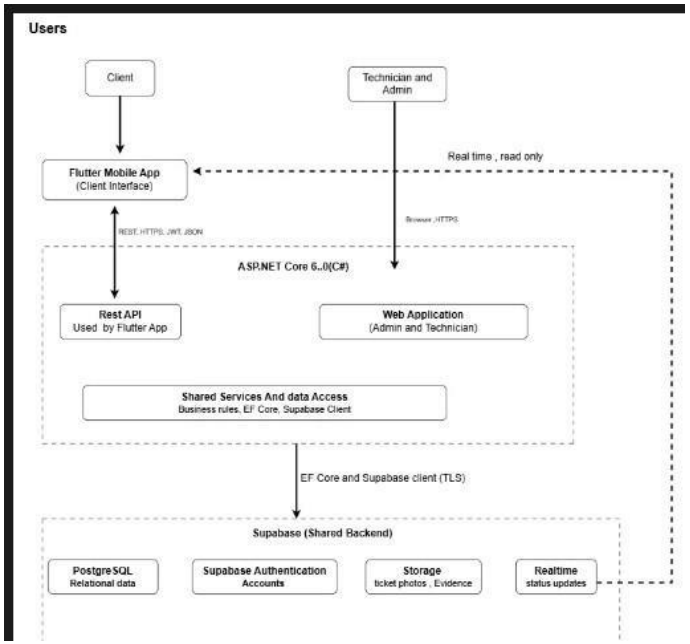
PHASE 2

System Architecture, UML, ERD and Interface Design

Unit 3

10. SYSTEM ARCHITECTURE DESIGN

10.1 System Architecture



Integration Explanation

Both the Flutter and ASP.NET interfaces use one ASP.NET layer that validates input and checks roles, and Supabase is the single source of data. A ticket submitted on a phone appears on the technician's dashboard, and the client sees the technician's update in the app.

Main Users / User Roles

1. **Client:** Registers, logs in, submits and tracks support requests using the Flutter interface.
2. **Technician:** Views and resolves assigned support requests using the ASP.NET web application.
3. **Admin:** Manages users, requests and assignments using the ASP.NET web application.

Flow

Client uses Flutter App → ASP.NET Core API → Supabase Database → Technician / Admin

Component Responsibilities

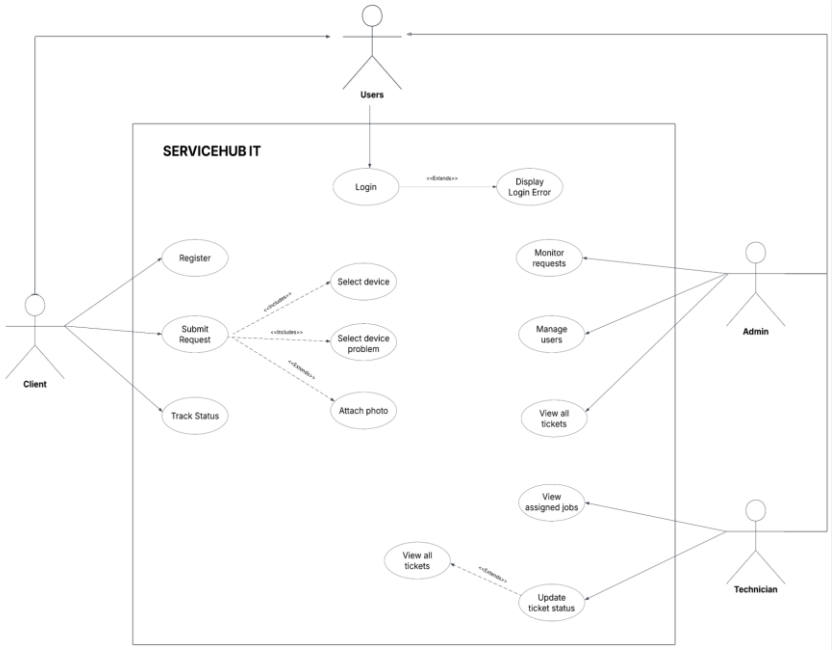
Flutter Mobile App: Client register and login, select device from approved menu, problem/service selection, request submission with description, request status tracking. It holds no database credentials, only the user's JWT.

ASP.NET Core Backend/API: Authentication and authorisation, request processing and validation, assignment of requests to technicians, status management, user management, communication handling, notification dispatch and API endpoints for Flutter and ASP.NET web clients.

ASP.NET Web App: The Technician and Admin interface for ticket management, assignment and user management. It shares one ASP.NET Core solution, and the same business rules, with the REST API used by Flutter.

Supabase Backend: PostgreSQL stores the data, Authentication issues JWTs, Storage holds ticket photos, and Realtime sends status updates.

10.2 Use Case



The diagram shows the system boundary "Service IT" and three actors: Client, Admin, and Technician.

Actors and Use Cases:

- **Client:** Register, Login, Submit Support Request, Track Request Status.
- **Admin:** Monitor requests, Manage technicians and users, Track or view status.
- **Technician:** View Assigned jobs and Update repair progress (which includes Upload diagnostics information and Record completed work).

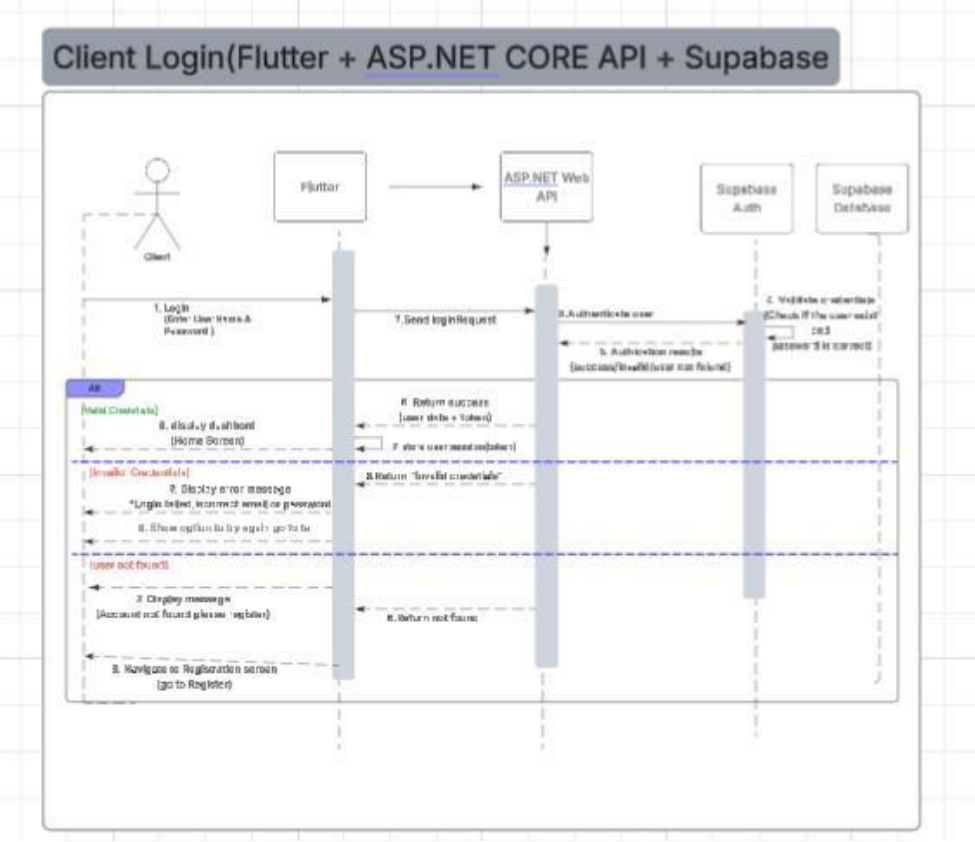
UML Relationships:

- <<include>>: Submit Support Request includes Select Device and Select Problem/Service. Update repair progress includes Upload diagnostics information and Record completed work.
- <<extend>>: Display Login Error extends Login. Describe problem extends Submit Support Request. View Repair status extends Track Request Status.

10.3 Sequence Diagrams

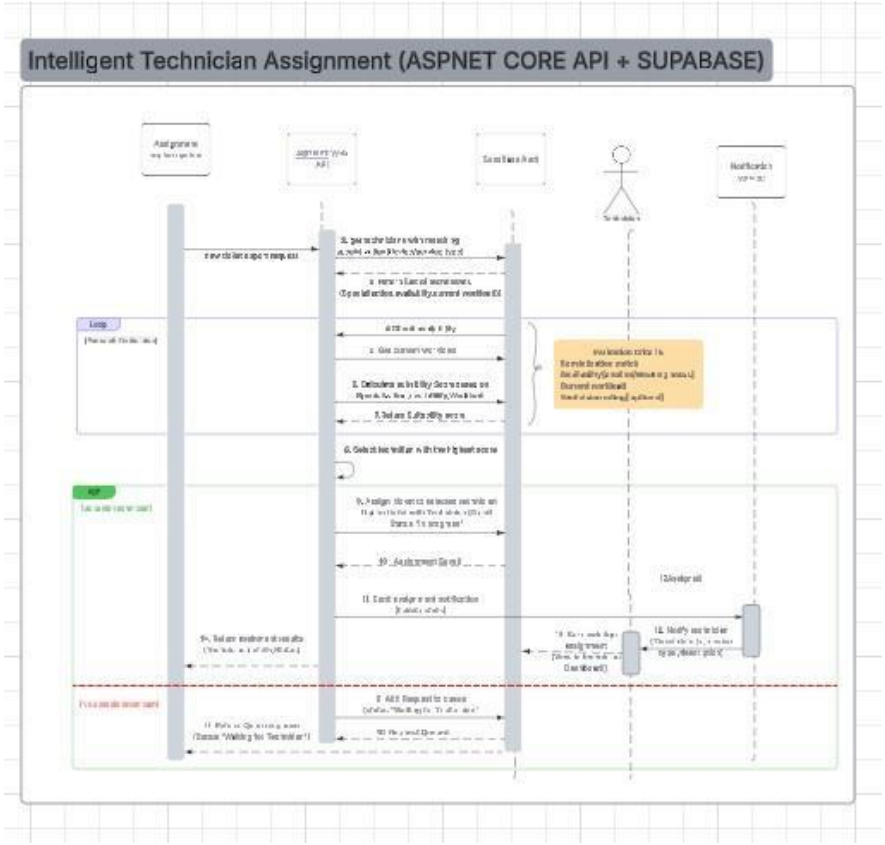
10.3.1 Sequence Diagram 1: Client Login (Flutter + Supabase)

Description: The client enters their email and password in the Flutter app, and the request goes to the ASP.NET API, which asks Supabase Authentication to verify the credentials. If they are valid, the client is shown the dashboard. If the password is wrong, an error message is shown with the option to try again. If the user is not found, the client is sent to the registration screen.



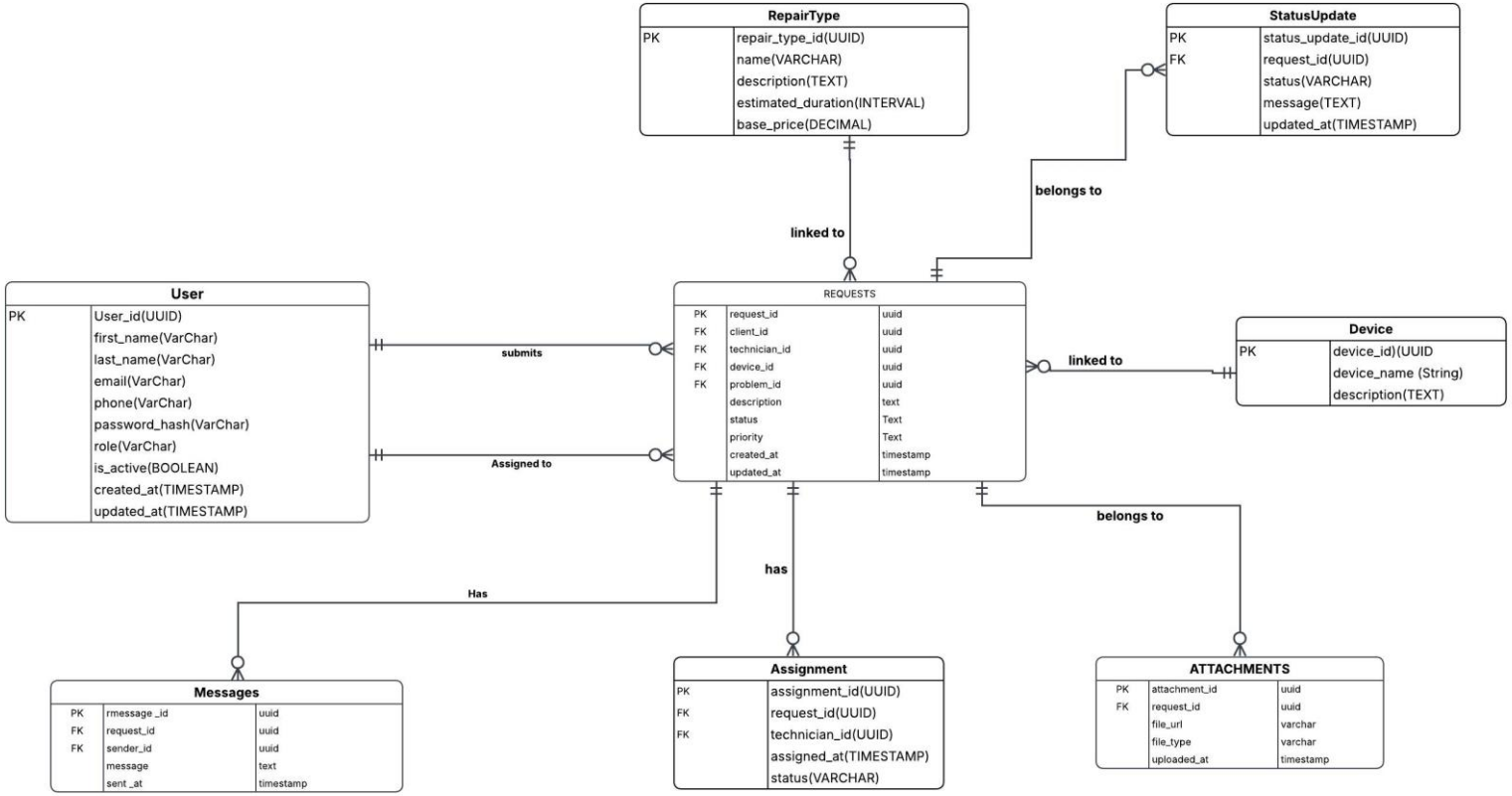
10.3.2 Sequence Diagram 2: Technician Assignment (ASP.NET + Supabase)

Description: When a new support request is submitted, the ASP.NET API reads the technicians from Supabase and scores each one, for example on skills, availability and current workload. The technician with the highest score is assigned and notified. If no suitable technician is available, the request waits until one is.



10.4 ERD

Description: The ERD shows how the system data is organised in the Supabase database. A User (client, technician or admin) submits and is assigned Requests. Each Request is linked to one Device and one RepairType, and can have many Messages, Attachments, StatusUpdates and Assignments.



Entity Purpose:

- **User:** Central table for all actors (Clients, Technicians, Admins). The role field controls access rights.
- **Device & RepairType:** The approved system catalogue. Defines what equipment is supported and what specific problems can be reported (FR2, FR3).
- **Requests:** The core hub linking the Client, Device, RepairType, and assigned Technician. Tracks the ticket status and priority (FR4, FR5, FR7).
- **Messages & Attachments:** Stores communication history and supporting photos/documents linked to a specific Request (FR6, FR11).
- **StatusUpdate & Assignment:** Audit trails logging status changes and technician assignments for transparency and accountability (FR5).

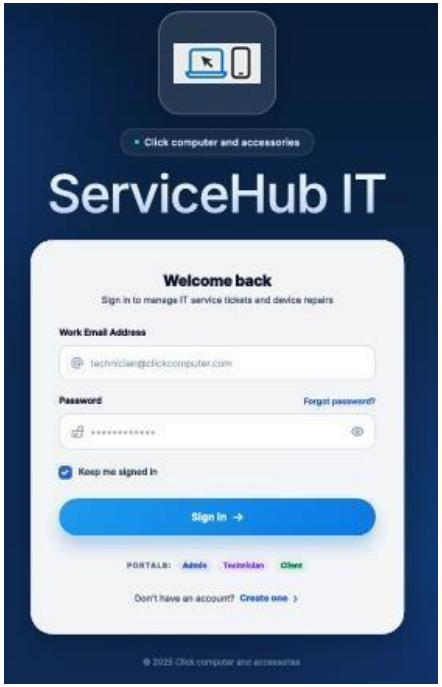
Key Relationships:

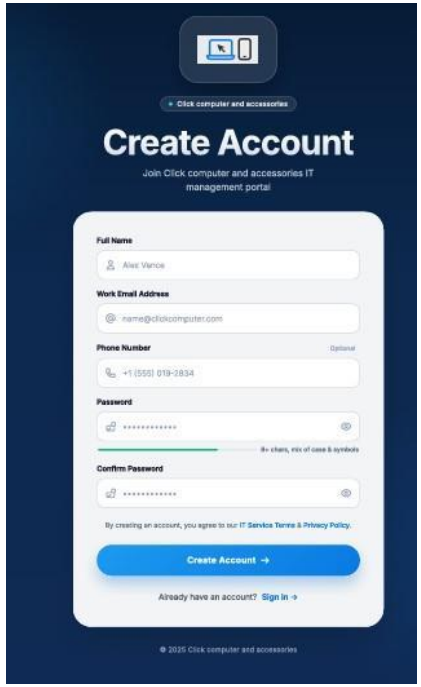
- **One-to-Many (User to Requests):** One Client can submit many Requests. One Technician can be assigned many Requests.
- **One-to-Many (Device/RepairType to Requests):** One Device type or RepairType can be linked to many Requests.
- **One-to-Many (Requests to Messages/Attachments/StatusUpdates/Assignments):** One Request can have multiple messages, multiple file attachments, multiple status history logs, and multiple assignment records.

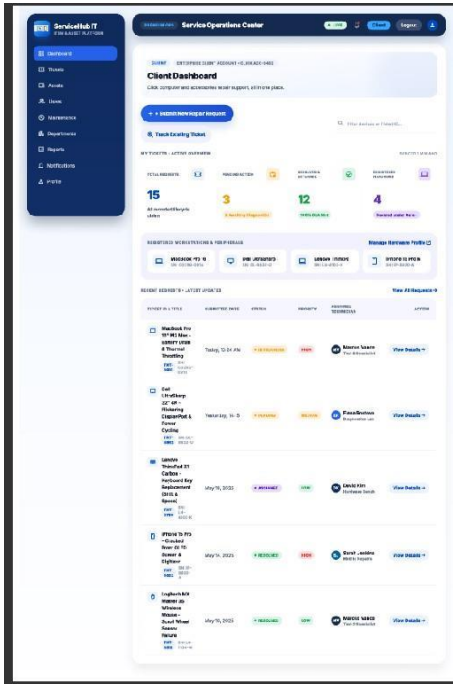
10.5 Flutter Mobile Interface Design

Description

Clients can register and log in, then are redirected to the Client dashboard page where they can track and submit repair requests.





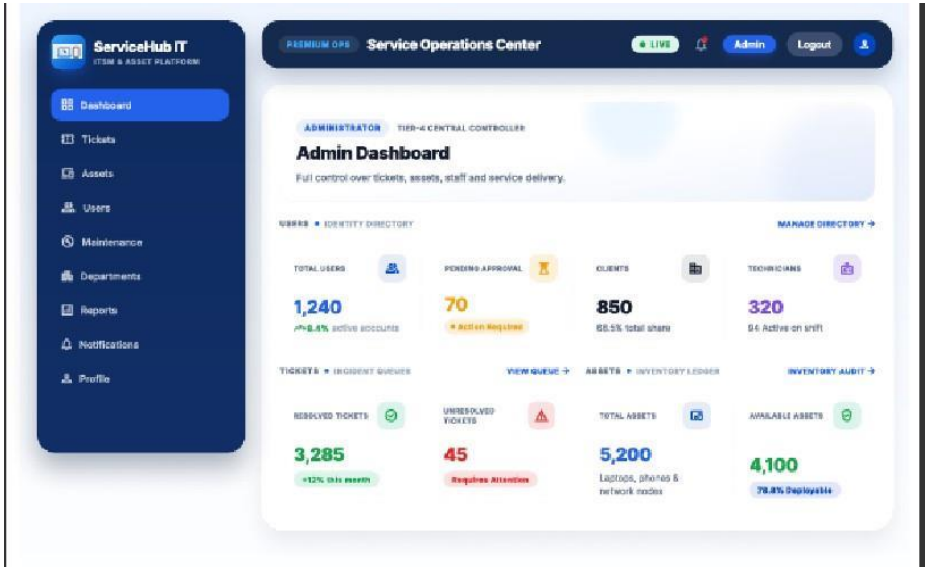


10.6 ASP.NET Web Interface Design

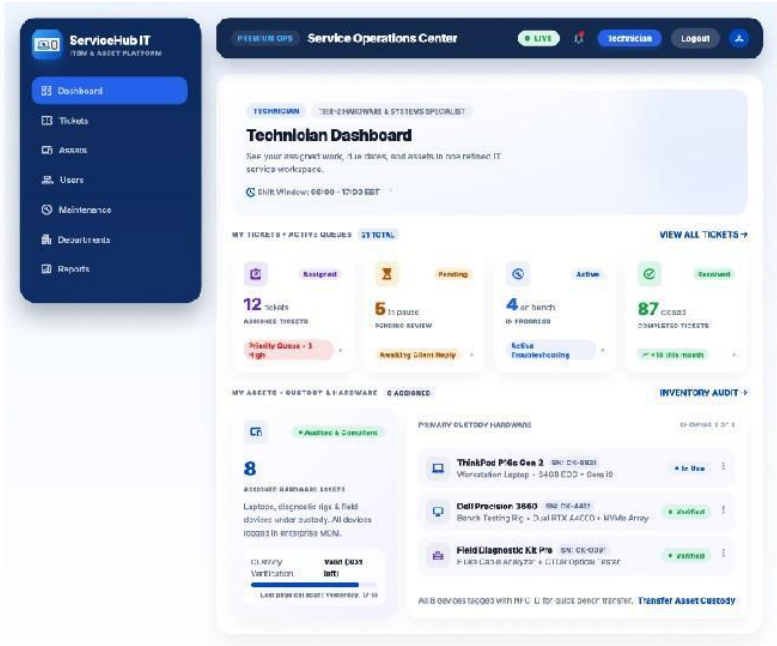
Admins and Technicians login then are redirected to designated screens.

Admins can view and monitor requests, manage users, and view all tickets.

Technicians can view the assigned jobs, record completed work and update status of progress.



Admin dashboard



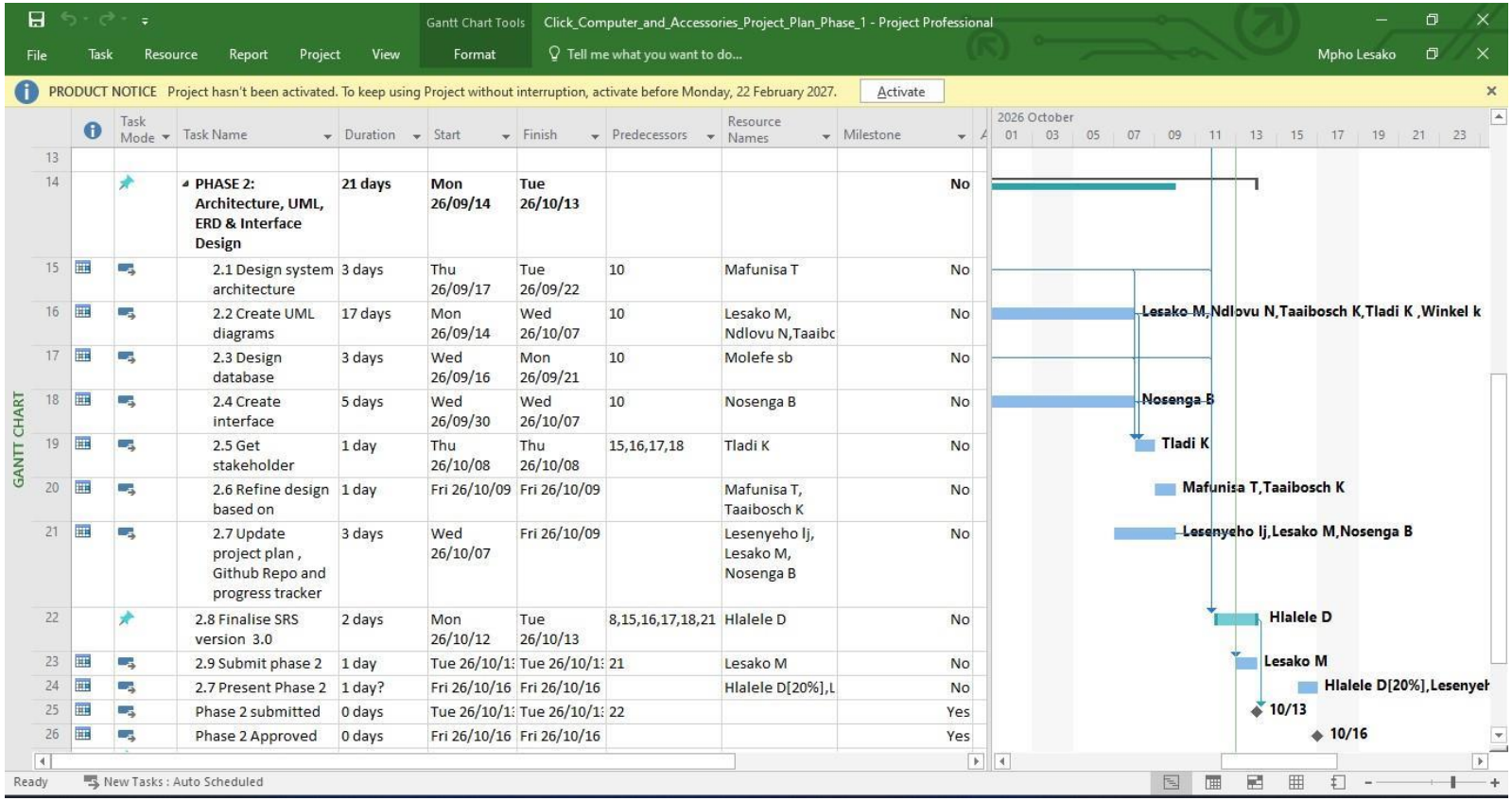
Technician dashboard

10.7 Design Alignment Table

Requirement	Use Case	Sequence Diagram	ERD table	Interface
FR1 Client Register/Login	Register/Login	Diagram 1 & 2	User	Flutter: Login screen
FR2 Device menu	Select device	Diagram 3	Device	Flutter: Device selection
FR3 Problem selection	Select problem	Diagram 3	RepairType	Flutter: Problem selection button
FR4 Submit request	Submit request	Diagram 3	Requests	Flutter: Request submitted
FR5 Track status	Track status	Diagram 6	Requests, User, Device, RepairType	Status tracking

Requirement	Use Case	Sequence Diagram	ERD table	Interface
FR7 Assign requests to technician	Manage requests	Diagram 4	Requests (technician_id), Assignment, User	Technician dashboard screen
FR8 Admin manages users, requests and assignment	Manage users	Diagram 7	User, Requests	ASP.NET Admin dashboard screen
FR10 Record device information	Select device	Diagram 3	Device, Requests	Flutter: Asset info field
FR11 Attach photo	Attach photo	Diagram 3	Attachments	Flutter: file Upload button
NFR1 Prevent unauthorised access	Login	Diagram 2	User (role)	All screens
NFR4 Clear navigation	-	-	-	Flutter UI (Navigation Flow)

10.8 Microsoft Project



	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Milestone
13								
14		**PHASE 2: Architecture, UML, ERD & Interface Design**	**21 days**	**Mon 26/09/14**	**Tue 26/10/13**			**No**
15		2.1 Design system architecture	3 days	Thu 26/09/17	Tue 26/09/22	10	Mafunisa T	No
16		2.2 Create UML diagrams	17 days	Mon 26/09/14	Wed 26/10/07	10	Lesako M, Ndlovu N,Taaibc	No
17		2.3 Design database	3 days	Wed 26/09/16	Mon 26/09/21	10	Molefe sb	No
18		2.4 Create interface	5 days	Wed 26/09/30	Wed 26/10/07	10	Nosenga B	No
19		2.5 Get stakeholder	1 day	Thu 26/10/08	Thu 26/10/08	15,16,17,18	Tladi K	No
20		2.6 Refine design based on	1 day	Fri 26/10/09	Fri 26/10/09		Mafunisa T, Taaibosch K	No
21		2.7 Update project plan , Github Repo and progress tracker	3 days	Wed 26/10/07	Fri 26/10/09		Lesenyeho lj, Lesako M, Nosenga B	No
22		2.8 Finalise SRS version 3.0	2 days	Mon 26/10/12	Tue 26/10/13	8,15,16,17,18,21	Hlalele D	No
23		2.9 Submit phase 2	1 day	Tue 26/10/1:	Tue 26/10/1:	21	Lesako M	No
24		2.7 Present Phase 2	1 day?	Fri 26/10/16	Fri 26/10/16		Hlalele D[20%],L	No
25		Phase 2 submitted	0 days	Tue 26/10/1:	Tue 26/10/1:	22		Yes
26		Phase 2 Approved	0 days	Fri 26/10/16	Fri 26/10/16			Yes

10.9 GitHub

GitHub Repository: https://github.com/tmafunisa24-sudo/ITC327W-Project

The repository contains the updated SRS, design files, and evidence of group activity.

10.10 Design Sources Link Table

Design	Tool	Source link
System architecture	Draw.io	https://drive.google.com/file/d/11Ja90b6tMq4ahkITHez3RUYbPcuNtIx/view?usp=sharing
Use cases	LucidCharts	https://lucid.app/lucidchart/895a8e0d-97e9-4649-a7c0-7f30062eb435/edit?viewport_loc=-7250%2C10394%2C4256%2C3450%2C0_0&invitationId=inv_13a5c578a00c4381-bcc7-15b1d5d87536

Design	Tool	Source link
Sequence diagram 1	LucidCharts	https://lucid.app/lucidchart/ec36d4bc-a084-4884-bbed-d6398301ee8f/edit?viewport_loc=-9560%2C2792%2C5464%2C2212%2C0_0&invitationId=inv_8c3b9894-ed5e-43c1-9f78-075a273945f1
Sequence diagram 2	LucidCharts	https://lucid.app/lucidchart/ec36d4bc-a084-4884-bbed-d6398301ee8f/edit?viewport_loc=-9560%2C2792%2C5464%2C2212%2C0_0&invitationId=inv_8c3b9894-ed5e-43c1-9f78-075a273945f1
ERD	LucidCharts	https://lucid.app/lucidchart/57c2cd7d-a52f-4d36-aba6-b10b1a96a3e2/edit?viewport_loc=-4225%2C1950%2C8513%2C6900%2C0_0&invitationId=inv_4eaa2ff6-389d-47f4-8048-3caf7578fdf2
Flutter wireframes	Figma	https://www.figma.com/design/IsuW8QCf5vvCxYrkFC2KOq/ServiceHubIT---Design-System---Prototypes?node-id=111-667&t=AklSjmxOGRMnR2Wq-1
ASP.NET wireframes	Figma	https://www.figma.com/design/IsuW8QCf5vvCxYrkFC2KOq/ServiceHubIT---Design-System---Prototypes?node-id=111-667&t=AklSjmxOGRMnR2Wq-1

11. EVIDENCE

- Client requirement and acceptance register is attached in the GitHub repository under evidence.

12. REVISION HISTORY

Version	Date	Changes
1.0	AUG 25	Initial SRS document
2.0	AUG 30	Added unit 2 feasibility, Risk register, refined scope and requirements
3.0	OCT 12	Added Unit 3 system design and architecture.
4.0	NOV 01	Added Unit 4 implementation and final prototype (Section 14), GitHub link and final Microsoft Project schedule.

13. APPROVALS

Role	Name	Date
Stakeholder	Morafo Prince	10 October 2026
Project Lead	Lesako Mpho	10 October 2026

PHASE 3

Implementation and Final Prototype

Unit 4

14.1 Overview of Implemented Components

The final prototype consists of a Flutter mobile application for clients, an ASP.NET Core 6.0 web application and API for administrators and technicians, and a shared Supabase backend (PostgreSQL, Authentication, Storage and Realtime). Clients register, log in, select a device and problem, and submit and track requests in the Flutter app, which communicates only with the ASP.NET API over HTTPS using JWTs. The ASP.NET layer validates input, enforces role-based access and reads and writes to Supabase, so a request submitted on a phone appears on the technician and admin dashboards, and status updates made there are reflected back in the client's app.

Component	Technology	What was implemented	Location in repository
Flutter mobile app	Flutter / Dart	Client register and login, device selection, problem selection, request submission, request status tracking	mobile/
ASP.NET web app and API	ASP.NET Core 8.0 / C#	Admin and technician dashboards, request monitoring, technician assignment, user management, status updates	web/
Supabase backend	PostgreSQL, Authentication, Storage, Realtime	Tables from the approved ERD, authentication and roles	database/

14.2 Shared Supabase Backend

- Tables implemented: profiles (users and their roles), tickets (support requests), ticket_attachments, assets, maintenance and service_catalog, with a department_id linking users and assets to departments. Some table names are different from the Phase 2 ERD .
- Every table has a uuid primary key. A ticket points to the user profiles through employee_id and technician_id and to an asset through asset_id, ticket_attachments are linked to tickets, and maintenance records link an asset to a technician. The diagram below shows these links.
- Authentication and roles and how permissions are enforced: We used Supabase's built-in security settings and checked the user's role (client, tech, or admin) when they log in to make sure they only see their own data.
- Both applications use the same Supabase project : The mobile app talks to our C# backend. The C# backend talks to Supabase. This means all the data is saved in one place.

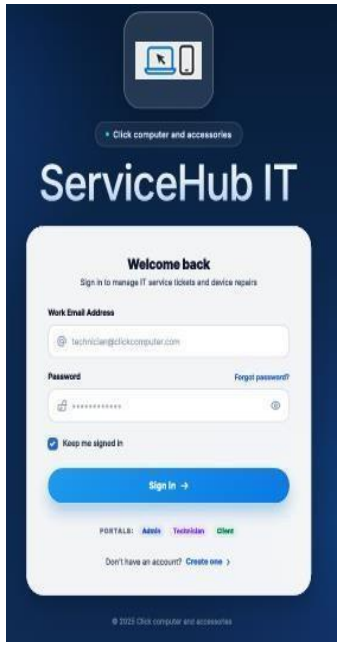
- **Dummy data used for the demonstration:** The database was populated with sample data so that every workflow could be demonstrated without real client information. This included test accounts for each role (one administrator, two technicians and several clients with fictional names and email addresses), assets for the approved devices, service catalogue entries for the problems that can be reported (e.g. screen damage, battery problems, software fault), and sample tickets in different statuses (Submitted, Assigned, In Progress, Resolved). No real personal data, passwords or credentials were used.
- Secret keys, passwords and private URLs are excluded from all screenshots and from the repository.



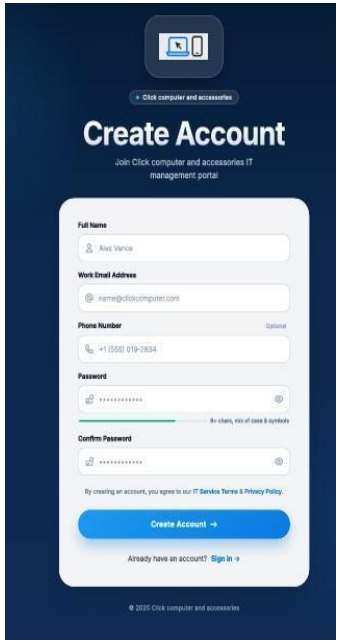
Supabase tables and relationships (part of the schema diagram)

14.3 Flutter Mobile Application

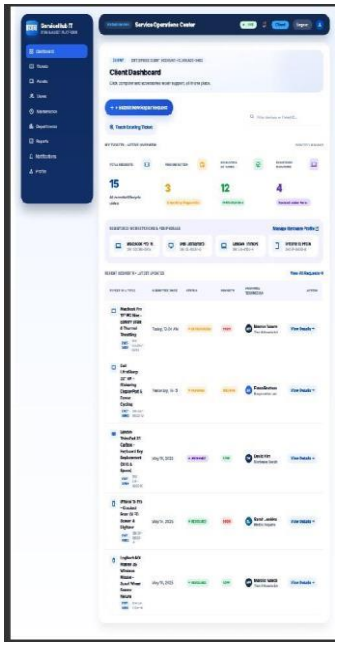
The Flutter app opens on the Login screen, with a link to Register for new users. After logging in, the client lands on the dashboard, from which they can submit a new repair request or track existing ones. Submission follows a short flow: select a device, select a problem, enter a description and submit. Input validation checks that the email format is valid, that passwords meet the minimum strength and match the confirmation field on registration, that required fields are not left empty, and that a device, problem and description are all provided before a request can be submitted. Clear error and confirmation messages are displayed (NFR2). Clients can only register, log in, submit requests, and view the status of their own requests. They cannot see other clients' requests or access any administrator or technician functions, and the app holds no database credentials, only the user's JWT.



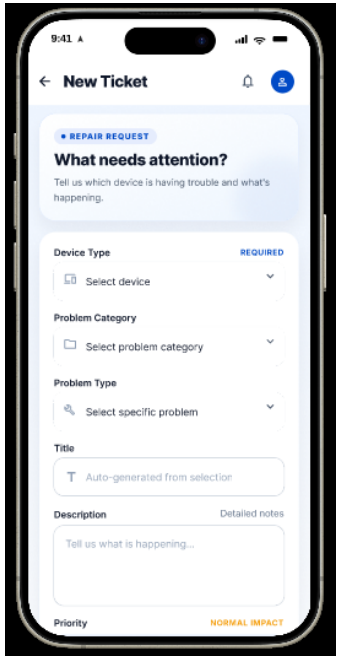
Login



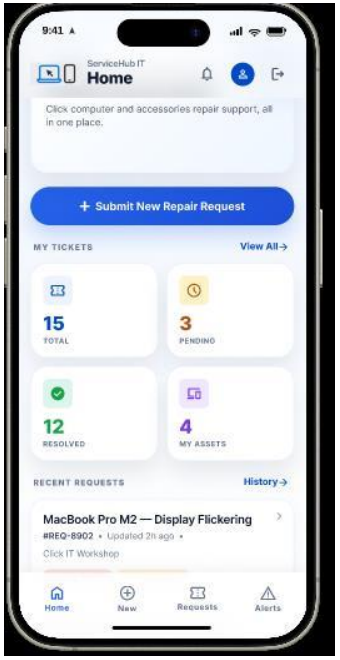
Register



Client dashboard



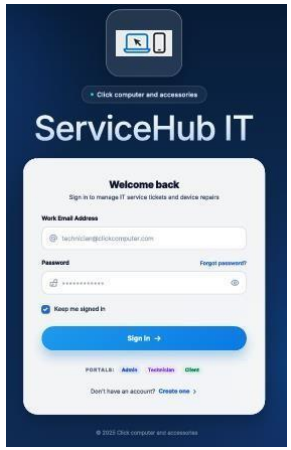
Select device/ enter problem



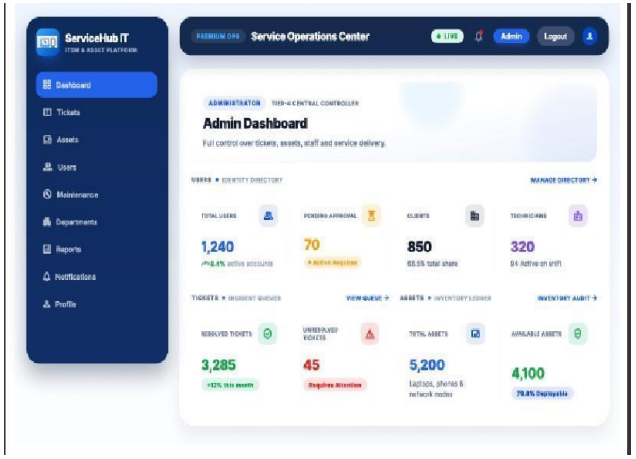
Track status

14.4 ASP.NET Web Application

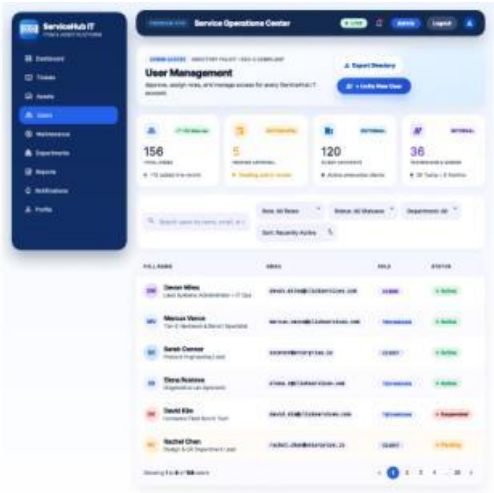
Administrators and technicians log in through a single web login and are redirected to their role-specific dashboard, with a sidebar for navigating between pages such as Dashboard, Tickets, Users and Reports. Input validation is performed on forms and again in the API, covering required fields, valid email format and permitted status values, and rejecting invalid data before it reaches the database. Access is role-based and enforced through JWT authentication. Administrators can monitor and view all requests, manage users and update request status. Technicians can only view the jobs assigned to them, record completed work and update the status of their requests and are blocked from accessing user management and administrator pages.



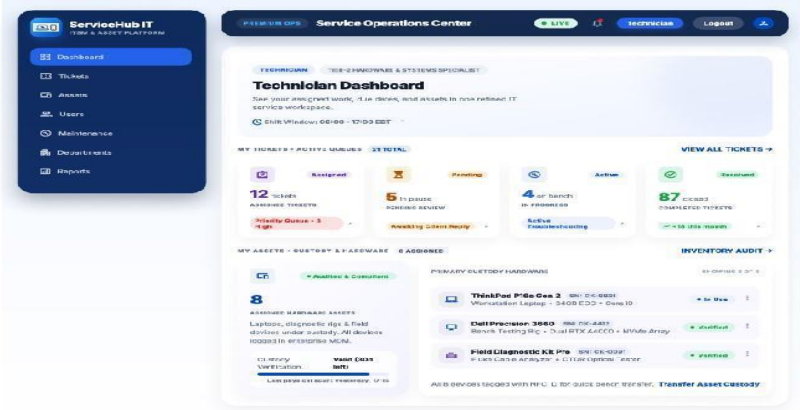
Login



Admin dashboard



User management



Technician dashboard/update status

14.5 Shared Processes

The following processes demonstrate Flutter and ASP.NET working with the same information through the shared Supabase backend.

	Process 1: Data submission and retrieval	**Process 2: Shared update and display**
User action	A client submits a support request in the Flutter app.	An authorised technician or administrator updates the request status in the ASP.NET web app.
Workflow	Flutter → ASP.NET API → Supabase (tickets) → ASP.NET dashboard	ASP.NET → Supabase (tickets) → Flutter app
Result	The new request appears on the ASP.NET admin/technician dashboard with the same values.	The client sees the updated status in the Flutter app.

14.6 Requirements-to-Implementation Table

Req. ID	Requirement	Implemented function	Application	Status
FR1	Register and log in	Registration and login with Supabase Authentication	Flutter	Working
FR2	Approved device menu	Device selection screen using the approved devices stored in Supabase	Flutter	Working
FR3	Problem/service options per device	Problem selection using the service catalogue in Supabase	Flutter	Working
FR4	Select problem, describe and submit request	Request form saved as a ticket in the tickets table	Flutter	Working
FR5	View request status	Status tracking screen	Flutter	Working
FR6	Client–staff communication	Not built. We did not make messaging screens in either app.	-	Incomplete
FR7	Assign requests to technicians	Assignment screen, which saves the chosen technician on the ticket (technician_id)	ASP.NET	Working
FR8	Admin manages users, requests, assignments	Admin dashboard and management pages	ASP.NET	Working
FR9	Notifications (Should Have)	Notifications page and bell icon in the mobile app, and a notifications page in the web app, read from Supabase.	Both	Working
FR10	Device/asset information (Should Have)	Assets pages in the web app record and show device and asset information (list, create, edit and details).	ASP.NET	Working
FR11	Attachments (Should Have)	Clients can attach a photo or file to a request, saved in the ticket_attachments table.	Flutter	Working
NFR1	Protect information from unauthorised access	Role-based access and JWT authentication	Both	Working

14.7 Approved Changes During Implementation

Change	Reason	Approved by / date
Device and asset information is recorded in the web app (Assets pages) and not on a Flutter screen.	Admin staff look after assets, and the mobile app is for clients only.	Group agreement, October 2026
The database uses ticketing-style table names (profiles, tickets, assets, maintenance, ticket_attachments, service_catalog) instead of the ERD names (User, Requests, Device, RepairType and so on).	The ticket workflow was built around tickets, assets and maintenance records, so the tables were named after those. The data it holds is the same as in the ERD.	Group agreement, October 2026

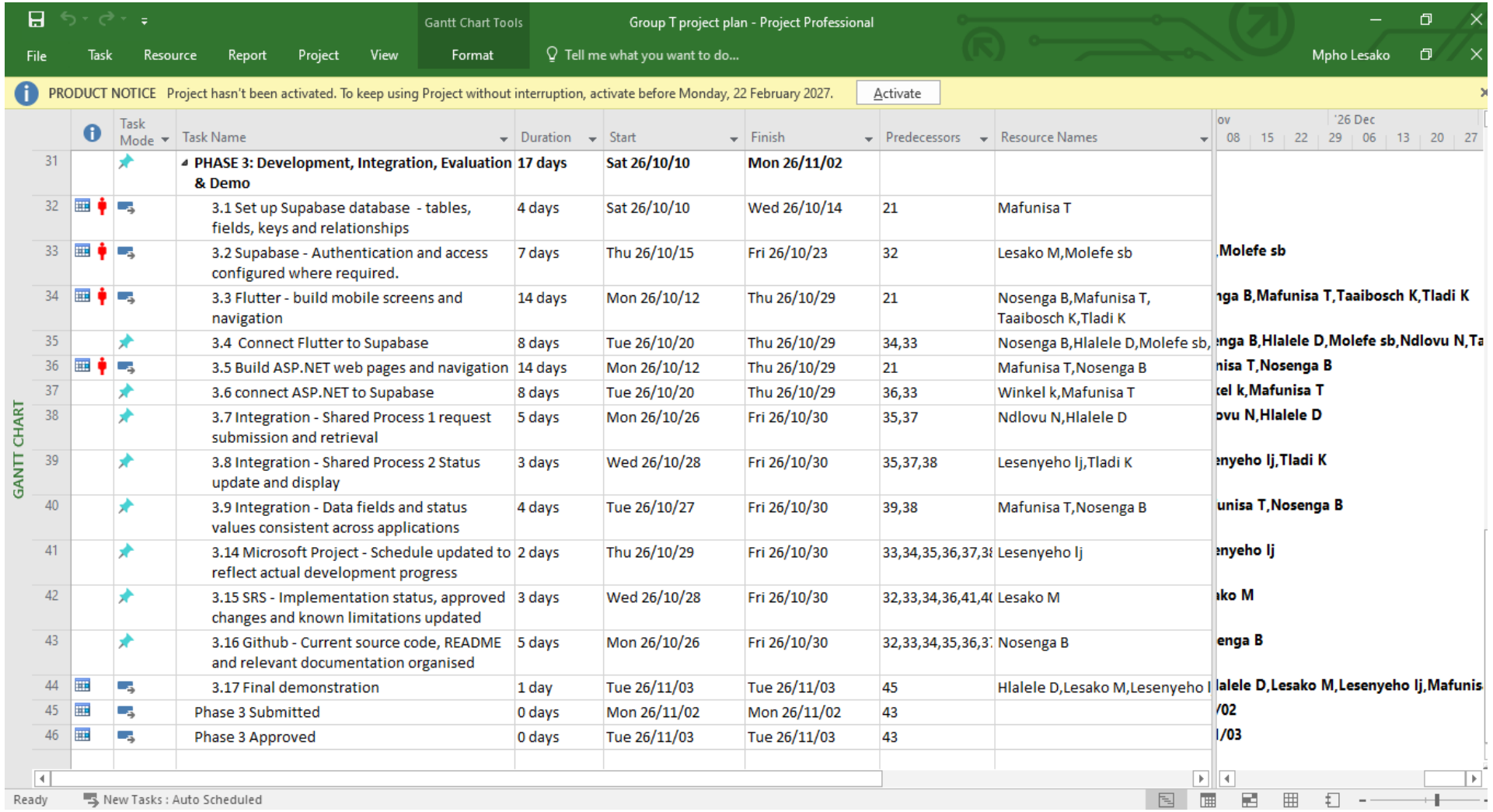
14.8 Known Limitations and Incomplete Functions

- Client and staff messaging (FR6) is not working. We did not build messaging screens in either app, so clients only see status changes and notifications.
- We did not write automated tests for the mobile app. Everything was tested by hand with dummy data.
- We did not measure the 2 second API response time (NFR5), and we only used the Supabase free tier, so the system has not been tried with many users at once.

14.9 GitHub Repository

- Repository: https://github.com/tmafunisa24-sudo/ITC327W-Project
- mobile/ – Flutter mobile application source code
- web/ – ASP.NET web application and API source code
- database/ – Supabase schema scripts and setup instructions
- README – basic setup and run instructions
- The submitted ZIP file and the GitHub repository contain the same version that was demonstrated.
- Original editable design source links remain available in Section 10.10.

14.10 Final Microsoft Project Schedule



Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names
31	**PHASE 3: Development, Integration, Evaluation & Demo**	**17 days**	**Sat 26/10/10**	**Mon 26/11/02**		
32	3.1 Set up Supabase database - tables, fields, keys and relationships	4 days	Sat 26/10/10	Wed 26/10/14	21	Mafunisa T
33	3.2 Supabase - Authentication and access configured where required.	7 days	Thu 26/10/15	Fri 26/10/23	32	Lesako M,Molefe sb
34	3.3 Flutter - build mobile screens and navigation	14 days	Mon 26/10/12	Thu 26/10/29	21	Nosenga B,Mafunisa T, Taaibosch K,Tladi K
35	3.4 Connect Flutter to Supabase	8 days	Tue 26/10/20	Thu 26/10/29	34,33	Nosenga B,Hlalele D,Molefe sb,
36	3.5 Build ASP.NET web pages and navigation	14 days	Mon 26/10/12	Thu 26/10/29	21	Mafunisa T,Nosenga B
37	3.6 connect ASP.NET to Supabase	8 days	Tue 26/10/20	Thu 26/10/29	36,33	Winkel k,Mafunisa T
38	3.7 Integration - Shared Process 1 request submission and retrieval	5 days	Mon 26/10/26	Fri 26/10/30	35,37	Ndlovu N,Hlalele D
39	3.8 Integration - Shared Process 2 Status update and display	3 days	Wed 26/10/28	Fri 26/10/30	35,37,38	Lesenyeho lj,Tladi K
40	3.9 Integration - Data fields and status values consistent across applications	4 days	Tue 26/10/27	Fri 26/10/30	39,38	Mafunisa T,Nosenga B
41	3.14 Microsoft Project - Schedule updated to reflect actual development progress	2 days	Thu 26/10/29	Fri 26/10/30	33,34,35,36,37,3	Lesenyeho lj
42	3.15 SRS - Implementation status, approved changes and known limitations updated	3 days	Wed 26/10/28	Fri 26/10/30	32,33,34,36,41,4	Lesako M
43	3.16 Github - Current source code, README and relevant documentation organised	5 days	Mon 26/10/26	Fri 26/10/30	32,33,34,35,36,3	Nosenga B
44	3.17 Final demonstration	1 day	Tue 26/11/03	Tue 26/11/03	45	Hlalele D,Lesako M,Lesenyeho l
45	Phase 3 Submitted	0 days	Mon 26/11/02	Mon 26/11/02	43	
46	Phase 3 Approved	0 days	Tue 26/11/03	Tue 26/11/03	43	

Milestone	Target date	Status
Backend implemented	01 November 2026	Complete
Flutter app completed	01 November 2026	Complete

Milestone	Target date	Status
ASP.NET app completed	01 November 2026	Complete
Integration and demonstration	03 November 2026	In Progress

14.11 Revision History Addendum

Version	Date	Changes
4.0	NOV 01	Added Unit 4 Implementation and Final Prototype section (Section 14), GitHub link and final Microsoft Project schedule.